B 3.1.2 Designing Classes

B 3.1.2 Construct a design of classes, their methods and behaviour.

This section covers software blueprint creation: defining classes, methods, and behaviors for specific applications. It also introduces UML class diagrams for visually representing and communicating software architecture during planning and design.

Introduction: Designing Classes, Methods, and Behaviour with UML

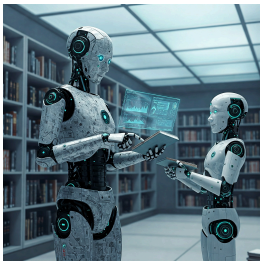
In the previous section, we explored the fundamental concepts of OOP, such as classes, objects, inheritance, encapsulation, and polymorphism. Now, we will delve into the crucial skill of designing these elements effectively.

This section focuses on how to construct a blueprint for your software by defining classes, their methods, and their behaviour based on the specific needs of an application.

We will also introduce Unified Modelling Language (UML) class diagrams as a powerful tool to visually represent these designs, aiding in both the initial planning and the communication of your software architecture.

PlantUML is a great tool for drawing diagrams from simple code. (https://www.planttext.com/)

Understanding Application Requirements and Their Impact on Class Design



Before we start drawing diagrams or writing code (which we will explore in detail later), the first critical step in OOP is to understand the application requirements you are building thoroughly.

- What problem are you trying to solve?
- What functionalities will the software need to provide?

Identifying these requirements will directly influence the classes you create, the attributes (data) they hold, and the methods (actions) they can perform.

Consider a scenario where you must design a simple library system. Some initial requirements might be:

- The system needs to keep track of books, including their titles, authors, and ISBNs.
- The system should allow users to borrow and return books.
- The system needs to record which user has borrowed which book.

Based on these basic requirements, we can start to consider the core entities involved. We'll likely need a way to represent a **Book** and a **User**. These real-world entities become the foundation for our classes in the OOP model.

From Requirements to Classes:

- The requirement to "keep track of books, including their titles, authors, and ISBNs" suggests that our `Book` class will need attributes to store this information.
- The requirement to "allow users to borrow and return books" implies that both the `Book` and `User` classes will need methods related to these actions. For instance, a `Book` might have a `borrow()` method, and a `User` might have a `borrowBook()` method.
- The requirement to "record which user has borrowed which book" might suggest a relationship between the `Book` and `User` classes, or perhaps even a separate class to represent a loan.

This initial analysis highlights how understanding the application's purpose directly shapes the fundamental building blocks of our OOP design: the classes themselves. Each class should have a clear responsibility within the system, contributing to the overall functionality required by the application.

Defining Classes and Their Methods

Once you understand the application requirements and have identified the key entities that need to be modelled, the next step is to define the classes. A class acts as a blueprint for creating objects. It specifies the attributes (data) that objects of that class will possess and the methods (behaviours or functions) that these objects can perform.

Components of a Class Design:

- **Class Name:** A descriptive name that clearly indicates the role of the class within the application (e.g., `Book`, `User`, `Loan`).
- **Attributes (Data Members):** These represent the state of an object. They are variables that hold information about the entity being modelled. For our `Book` class, attributes might include `title` (string), `author` (string), and `isbn` (string).
- **Methods (Member Functions):** These define the behaviour or actions that objects of the class can perform. For our `Book` class, methods might include `borrow()`, `return()`, and `getDetails()`. For a `User` class, methods could include `borrowBook(Book book)`, `returnBook(Book book)`, and `getBorrowedBooks()`.

Designing Methods Based on Application Needs:

The methods you define for a class should directly address the required functionalities of the application. Consider the `borrow()` method in our `Book` class. What should this method do?

- It might need to check if the book is already borrowed.
- If not, it should update the book's status to "borrowed" and perhaps record which user borrowed it.

Similarly, the `borrowBook(Book book)` method in the `User` class would need to:

- Check if the user can borrow more books (perhaps there's a borrowing limit).
- If so, it must associate the given `Book` object with this `User` object.

Thinking through the steps involved in each required action helps define the necessary methods and their parameters.

Introduction to Unified Modelling Language (UML) Class Diagrams

As your software design becomes more complex, it's essential to have a way to visually represent the classes, their attributes, their methods, and the relationships between them. This is where UML class



diagrams come in. UML is a standardised modelling language used in software engineering to create visual representations of a system. Class diagrams are one of the most common and useful types of UML diagrams for OOP design.

Purpose of UML Class Diagrams:

- **Visual Communication:** They provide a clear and concise visual representation of the static structure of a system, making it easier for developers and stakeholders to understand the design.
- **Design Aid:** Creating class diagrams helps you think through your system's design before you start coding, identifying potential issues and areas for improvement early on.
- **Documentation:** Class diagrams serve as valuable documentation of your software architecture, making it easier to maintain and evolve the system over time.
- **Planning:** They help plan the development process by outlining the components that need to be created and how they interact.

Key Elements of a UML Class Diagram

A UML class diagram typically consists of one or more rectangles, each representing a class. These rectangles are divided into three compartments:

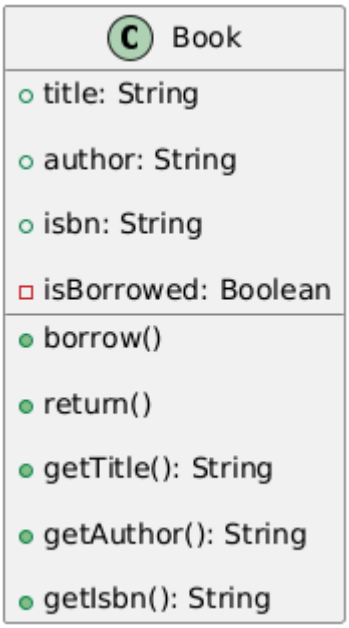
1. **Top Compartment:** Contains the name of the class. Class names are usually nouns and are written in Title Case (e.g., `Book`, `User`).
2. **Middle Compartment:** Lists the attributes (data members) of the class. Each attribute is typically shown with its visibility, name, and type optionally.
3. **Bottom Compartment:** Lists the methods (member functions) of the class. Each method is typically shown with its visibility, name, parameters (including their types), and optionally, its return type.

Visibility:

Visibility symbols indicate the access level of attributes and methods:

- + (Public): Accessible from any other class.
- - (Private): Accessible only within the class itself. This supports the principle of encapsulation.
- # (Protected): Accessible within the class itself and by its subclasses (relevant when discussing inheritance).
- ~ (Package/Default): Accessible within the same package (the default if no visibility is explicitly stated).

Example UML Class Diagram for the `Book` Class:




```
@startuml
class Book {
  + title: String
  + author: String
  + isbn: String
  - isBorrowed: Boolean
  + borrow()
  + return()
  + getTitle(): String
  + getAuthor(): String
  + getIsbn(): String
}
@enduml
```

In this diagram:

- `Book` is the name of the class.
- `title`, `author`, and `isbn` are public attributes of type `String`.
- `isBorrowed` is a private attribute of type `Boolean`, indicating whether the book is currently checked out.
- `borrow()`, `return()`, `getTitle()`, `getAuthor()`, and `getIsbn()` are public methods. `getTitle()`, `getAuthor()`, and `getIsbn()` are shown with their return type `String`. The `borrow()` and `return()` methods don't explicitly show a return type here, implying they might not return a value (or return a void type).

Representing Class Relationships in UML

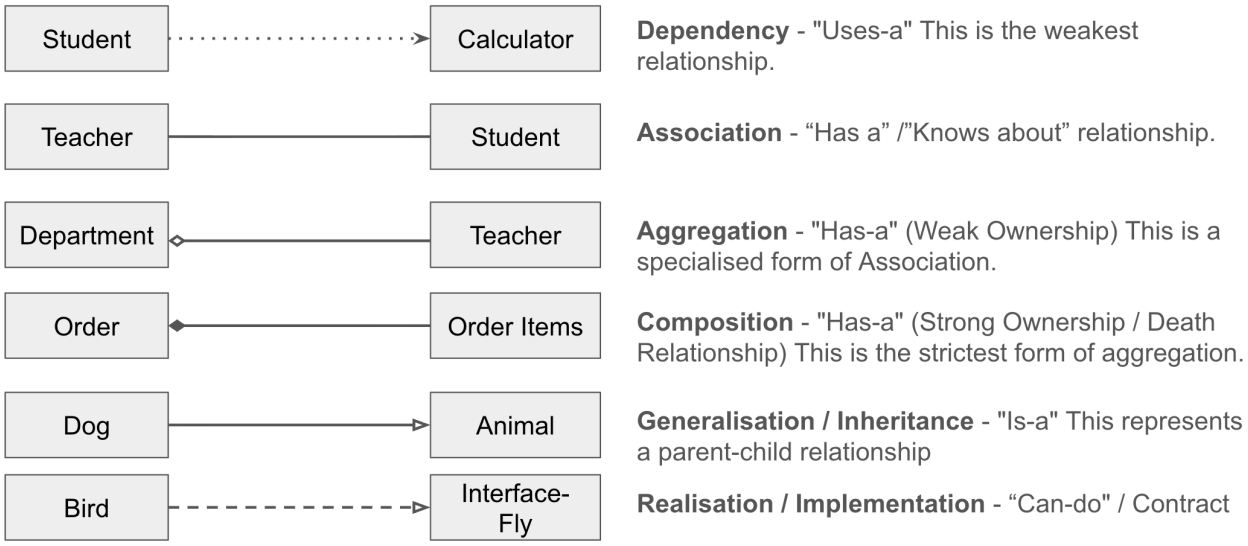
UML class diagrams are particularly powerful for showing how different classes in a system relate to each other. Common relationships include:

- **Association:** A general relationship between classes, indicating that objects of these classes can interact with each other. A solid line between the classes often represents associations. You can add

multiplicity symbols to the ends of the line to indicate how many objects of each class are involved in the relationship (e.g., 1, *, 0..*, 1..*). You can also add a label to the association line to describe the nature of the relationship.

- **Dependency:** A “uses-a” relationship that is a weaker form of association where one class uses another class. A dependency is represented by a dashed arrow pointing from the dependent class to the class it uses.
- **Aggregation:** A "has-a" relationship representing a whole-part relationship, where the part can exist independently of the whole. Aggregation is represented by a solid line with an open diamond at the end connected to the whole class.
- **Composition:** A stronger form of aggregation ("has-a") representing a whole-part relationship where the part cannot exist independently of the whole. Composition is represented by a solid line with a filled diamond at the end connected to the whole class.
- **Inheritance (Generalisation):** Represents an "is-a" relationship where one class (the subclass or derived class) inherits properties and methods from another class (the superclass or base class). Inheritance is represented by a solid line with a hollow arrowhead pointing from the subclass to the superclass.

All the relationships possible can be seen below. These are covered in more detail in later lessons.



UML relationships

UML as an Aid to Effective Software Design and Planning

Creating UML class diagrams is not just about drawing pretty pictures. It's a fundamental step in effective software design and planning.

Benefits of Using UML in Design and Planning:

- **Early Error Detection:** By visually modelling your system, you can identify inconsistencies, missing elements, or problematic relationships early in the development process, before writing any code. For example, you might need an attribute in a class you initially overlooked.
- **Improved Communication:** UML diagrams provide a common language for discussing the system's design among team members, stakeholders, and even yourself at a later stage. This ensures everyone has a shared understanding of the software's architecture.
- **Better Organisation:** Modelling forces you to think systematically about your application's different components and how they fit together. This promotes a more organised and structured approach to software development.

- **Facilitates Reusability:** A well-designed class diagram can highlight opportunities for code reuse through inheritance and other relationships.
- **Supports Iterative Development:** UML diagrams can be created and refined iteratively as your understanding of the requirements evolves. You can start with a high-level overview and gradually add more detail.
- **Foundation for Implementation:** The class diagram is a direct guide for the subsequent coding phase, outlining the classes you need to create, their attributes, and their methods.

In the IB Computer Science course, using UML class diagrams is an essential skill for planning and documenting your solutions, particularly in the internal assessment (IA), where you must clearly communicate the design of your computational solution.